This repository has been archived by the owner. It is now read-only.

Whinarn / MeshDecimator

Public archive

Watch 24 Star 251 Fork 37

Code Issues Pull requests Actions Projects Wiki Security Insights

master 4 branches 4 tags Go to file Add file Code

Whinarn chore: update readme about project archiving a1e85d4 on Aug 28, 2020 134 commits

MeshDecimator	Updated the changelog and assembly versions in preparation for the re...	3 years ago
MeshDecimatorTool	fixed texture index wrong parsing and generated mtl file when materia...	2 years ago
UnityExample	Updated the MeshDecimator assembly binaries in the Unity example.	3 years ago
.gitattributes	Updated the precompiled assemblies in the Unity example project.	4 years ago
.gitignore	Uploaded the initial projects with solution.	5 years ago
CHANGELOG.md	Updated the changelog and assembly versions in preparation for the re...	3 years ago
LICENSE.md	Updated the copyright notice year.	4 years ago
MeshDecimator.sln	The solution file is now for Visual Studio 2017.	4 years ago
README.md	chore: update readme about project archiving	14 months ago

README.md

MeshDecimator (Archived)

A mesh decimation library for .NET and Unity. The project is written entirely in C# and released under the MIT license.

WARNING: This repository has been archived in favor of UnityMeshSimplifier, which includes a more updated version of this algorithm for the Unity game engine. This means that I'm dropping the .NET support outside of Unity due to not having the time to maintain two libraries that includes almost the same code. Anyone who wishes to continue on this project is free to do so under the current license.

Compatibility

This provided Unity Example project is currently compatible with Unity 2018.1.2f1

This does not in any way limit the code to that specific Unity version however, but you might have to modify some Unity API usage.

Prerequisites

The following must be installed before this repository can be properly cloned:

- git-lfs

Installation into Unity

After building the project, copy over *MeshDecimator.dll* and *MeshDecimator.xml* into your Unity project, anywhere within your Assets directory (for example *Assets/MeshDecimator*). It is strongly recommended to build the C# project over copying the binaries found in the Unity Example project.

In the provided Unity example in this repository you can find examples of how to use the mesh decimation from Unity.

Fast Quadric Mesh Simplification Algorithm

MeshDecimator uses an algorithm based on the Fast Quadric Mesh Simplification algorithm, completely rewritten in C#. Currently it is the only mesh decimation algorithm available to use.

The Smart Linking feature

In order to solve artifacts in the mesh simplification process where holes or other serious issues could arise, a new feature called smart linking has been introduced. This feature is enabled by default but can be disabled through the *EnableSmartLink* property on the *FastQuadricMeshSimplification* class. Disabling this could give you a minor performance gain in cases where you do not need this.

The *VertexLinkDistanceSqr* property on the *FastQuadricMeshSimplification* class could be used to change the maximum squared distance between two vertices for the linking. The default value is *double.Epsilon*.

My decimated meshes have holes, what is wrong?

The original algorithm that was ported from C++ did not support situations where multiple vertices shared the same position, instead of being treated as one vertex they were treated individually. This would then end up creating visible holes in the mesh where the vertices were not connected through triangles.

There are several ways to solve this problem. The smart linking feature (mentioned above) is enabled by default and should take care of most of these problems for you. But there are also options to preserve borders, seams and UV foldovers. The properties *PreserveBorders*, *PreserveSeams* and *PreserveFoldovers* will preserve some vertices from being decimated, strongly limiting the decimation algorithm, but should prevent holes in most situations.

The recommendation is to use the smart linking feature that is enabled by default, but the options for preservation exists in those cases where you may want it.

Credits

The teddy bear model in the Unity example project is from the following package from Unity Asset Store:
https://assetstore.unity.com/packages/essentials/tutorial-projects/mecanim-example-scenes-5328

About

A mesh decimation library for .NET and Unity.

unity dot-net mesh-simplification mesh-decimation-library mesh-decimation

Readme

MIT License

Releases

4 tags

Packages

No packages published

Contributors 2

Whinarn Mattias Edlund

XinJack

Languages

C# 100.0%